Approximating the Sine Function on FPGA

IEEE 754 Single-Precision, Pipelined AXI4-Stream Implementation

Student: Diaconu Mihai
Supervising professor: Lazea Dragos Bogdan

Technical University of Cluj-Napoca
Faculty of Automation and Computer Science

October 19, 2026

Contents

1 Introduction **2**
1.1 Problem description 2
1.2 Motivation 2
1.3 Objectives 3

2 Bibliographic Study **4**
2.1 Theoretical background 4
2.1.1 IEEE-754 single-precision floating point 4
2.1.2 Elementary function evaluation 5
2.1.3 AXI-4 Streaming interface 5
2.2 Existing approaches for approximations 6
2.2.1 Table-based methods 6
2.2.2 Shift-and-add algorithms (CORDIC) 6
2.2.3 Polynomial approximation 6
2.2.4 Bit-manipulation techniques 6
2.2.5 Vendor IP cores 6
2.3 Work plan 6

Chapter 1

Introduction

1.1 Problem description

The evaluation of non-linear elementary functions, in particular trigonometric functions such as the sine function, $\sin(x)$, is a frequent operation in digital systems. Applications such as machine learning and digital signal processing can often make do with fast, approximate evaluations of these functions [1]. Muller [1] also reviews hardware designs built for this purpose, such as a table-based sine approximation proposed for a spread-spectrum frequency synthesizer and an elementary-function accelerator for a neuromorphic chip. This project considers real-valued inputs represented in the 32-bit single-precision (binary32) floating-point format defined by the IEEE 754 standard [2].

1.2 Motivation

Elementary functions such as $\sin(x)$ cannot be computed exactly with a finite number of arithmetic operations. Any implementation, either software or hardware, can only approximate them [1]. The precision of the result therefore becomes a design decision. General-purpose implementations aim for results as close as possible to the exact value, while many applications can tolerate a less accurate result if it is obtained faster [1].

Choosing how to make this trade-off is not straightforward. In his review of approximation techniques, Muller [1] does not recommend a single method: the most suitable one depends on the function being evaluated, the required accuracy, the implementation technology and the memory available for tables. He also notes that the methods differ in versatility, accuracy and scalability, which makes a general comparison difficult.

On a Field-Programmable Gate Array (FPGA), this trade-off can be measured directly. Each approximation method has a concrete cost in look-up tables (LUTs), DSP slices, block RAM, latency and maximum clock frequency, and the best choice can change with the required precision. For example, Muller reports work in which CORDIC proved to be a good candidate for low-precision FPGA implementations of the arctangent function [1].

General guidance from the literature is therefore not enough to select a method for a specific platform and number format. A reliable choice requires the candidate methods to be implemented on the same device and measured under identical conditions, so that their accuracy and hardware cost can be compared directly, including against the solutions already offered by the FPGA vendor.

1.3 Objectives

The main objective of this project is to design, implement and evaluate two pipelined hardware architectures that compute the sine of IEEE 754 single-precision floating-point numbers on an FPGA, exposed through AXI4-Stream interfaces, and to compare their accuracy, resource usage and latency with the Xilinx IP-cores available in the Vivado design suite.

The methods will be described in synthesizable VHDL, so that they can be implemented on a real device and validated in hardware on a Digilent Nexys A7 development board, in addition to being verified in simulation.

Chapter 2

Bibliographic Study

2.1 Theoretical background

2.1.1 IEEE-754 single-precision floating point

Before going into the methods for approximating the sine function, we need to understand the way real numbers are represented in memory using the IEEE 754 single-precision floating point standard [2].

The standard defines arithmetic formats, rounding rules, operations and exception handling [2]. For this project specifically we are interested in the specifics of **binary32**. As its name suggests, the **binary32** uses 32 bits to represent a real number:

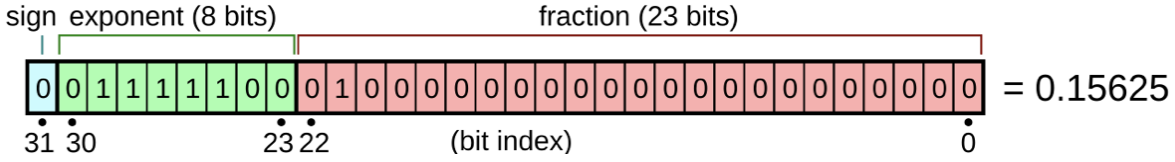


Figure 2.1: The IEEE 754 binary32 (single-precision) format [3]

The most significant bit gives the sign of the number - 0 for positive, 1 for negative values. Next 8 bits represent the exponent, biased with 127. The rest of 23 bits represent the mantissa (or fraction) which are the fractional part of the number with the implicit leading 1 bit for normalized values [1, 3]. The standard describes special exponents to represent special values like ± 0, NaN, $\pm\infty$.

The exponent values $E = 0$ and $E = 255$ are reserved for special values, as summarized in table 2.1 [3].

Table 2.1: Special values in the binary32 format

Exponent E	Fraction f	Value
0	0	± 0
0	$\neq 0$	subnormal: $(-1)^s \times 2^{-126} \times 0.f$
255	0	$\pm\infty$
255	$\neq 0$	NaN (not a number)

For a normalized number, with sign bit b_{31}, biased exponent E stored as an 8-bit unsigned integer and a 23-bit fraction, the represented value is

$$(-1)^{b_{31}} \times 2^{(b_{30}b_{29}\ldots b_{23})_2 - 127} \times (1.b_{22}b_{21}\ldots b_0)_2 \tag{2.1}$$

which can also be written as a sum over the fraction bits [1, 3]:

$$\text{value} = (-1)^{\text{sign}} \times 2^{(E-127)} \times \left(1 + \sum_{i=1}^{23} b_{23-i}\, 2^{-i}\right) \tag{2.2}$$

2.1.2 Elementary function evaluation

Following [1], the term *elementary functions* refers to the most widely used mathematical functions: sin, cos, tan, $\sin^{-1}$, $\cos^{-1}$, $\tan^{-1}$, sinh, cosh, tanh, $\sinh^{-1}$, $\cosh^{-1}$, $\tanh^{-1}$, as well as exponentials and logarithms in bases 2, e and 10. These elementary functions can only be approximated, they cannot be computed exactly with a finite number of operations, as mentioned in section 1.2.

Measuring accuracy. Since an elementary function can only be approximated, the quality of an implementation is described7 by the error between the computed result y^* and the exact result $y = f(x)$. Two measures are commonly used: the *absolute error*, $|y^* - y|$, and the *relative error*, $|y^* - y|/|y|$. The ideal behaviour is *correct rounding*, in which the implementation always returns the floating-point number nearest to the exact result [1].

2.1.3 AXI-4 Streaming interface

In a dataflow architecture, processing is organized as a pipeline of independent modules, where each module receives data from the previous one and passes its results to the next. Since modules may need a different number of clock cycles to process a data item, they rarely run at the same rate: a fast sender feeding a slower receiver must stall until the receiver can accept new data, so the throughput of the whole pipeline is bottlenecked by its slowest stage. Moreover, if a sender kept transmitting while its receiver was busy, data would be lost. A flow control mechanism is therefore needed between the stages [4].

The AXI4-Stream protocol is a great interface to connect modules that need to exchange data. It connects a sender and a reciever together relying on a **ready/valid** handshake principle.

The interface consists of 5 main signals: a shared **CLOCK**, a global **RESET** (active LOW), the **DATA** lines, and 2 "control" bits: **VALID** (from the sender to the reciever) and **READY** (from the reciever to the sender) [4]. Of course, the interface provides other optional signals, but the ones mentioned above are mandatory for reliable transfer of data between the modules.

Linking two modules together is simply done by connecting the output of the "sender" module to the inputs of the "reciever" module.

The core HANDSHAKE rules. As mentioned above, the **TVALID** and **TREADY** signals dictate when the information is transmitted across. This two-way mechanism allows both the sender and the reciever to control when the information is passed. The AXI4-Stream protocol imposes a very important rule, that is, for a data transfer to

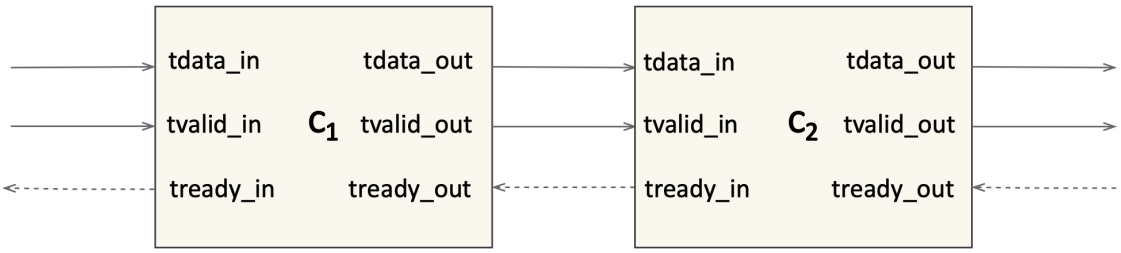


Figure 2.2: Two interconnected components using AXI4-Stream Interface [4]

happen, both **TVALID** and **TREADY** signals must be asserted during the same clock cycle [4]. Am ramas la handshake.

2.2 Existing approaches for approximations

2.2.1 Table-based methods

2.2.2 Shift-and-add algorithms (CORDIC)

pe foaie deocamdata

2.2.3 Polynomial approximation

2.2.4 Bit-manipulation techniques

2.2.5 Vendor IP cores

2.3 Work plan

Table 2.2: Targets for each project meeting

Meeting	Target
PM 1	Pick the project and begin studying
PM 2	Introduction and bibliographic study
PM 3	Initial design for the two approximation methods
PM 4	Updated design of the two approximation methods
PM 5	Start implementing methods in hardware
PM 6	Continue implementing in hardware and start testing
PM 7	Finish testing and present final project

Bibliography

[1] J.-M. Muller, "Elementary functions and approximate computing," *Proceedings of the IEEE*, vol. 108, no. 12, pp. 2136–2149, 2020. DOI: 10.1109/JPROC.2020.2991885. HAL: hal-02517784v2.

[2] Wikipedia, "IEEE 754." Wikipedia, The Free Encyclopedia, 2026. Accessed: Oct. 2026. Available: `https://en.wikipedia.org/wiki/IEEE_754`.

[3] Wikipedia, "Single-precision floating-point format." Wikipedia, The Free Encyclopedia, 2026. Accessed: Oct. 2026. Available: `https://en.wikipedia.org/wiki/Single-precision_floating-point_format`.

[4] Structure of Computer Systems, "Laboratory 4: Dataflow architectures. AXI4-Stream standard." Laboratory guide, Technical University of Cluj-Napoca, 2026.